WEEK 04 — Data Representation, Integer & Floating-Point Representation

Complete Notes
English Only
Theory + Numericals

Week 4 is one of the most important weeks for your Computer Organization midterm because it contains several numerical topics, especially:

- Signed and unsigned integers
- Sign bit
- 1's complement
- 2's complement
- Integer range
- Overflow
- Sign extension and zero extension
- Fixed-point representation
- Floating-point representation
- IEEE 754
- ASCII and Unicode

We will cover every topic step by step, including solved numerical questions and expected exam questions.

PART 1: Data Representation

1. What is Data Representation?

Definition:

Data representation is the method used by a computer to store, process, and represent different types of information in binary form (0s and 1s).

Explanation

Computers understand only binary digits, which are 0 and 1.

However, we work with different types of information, such as:

1. Positive and negative numbers.
2. Decimal numbers with fractional parts.
3. Characters and symbols.

4. Images and audio.
5. Instructions and program data.

Therefore, computers use different representation techniques to store this information.

Examples

Data Type	Representation
Positive integers	Unsigned binary
Positive and negative integers	Signed binary
Negative integers	Usually two's complement
Decimal fractions	Floating-point or fixed-point
Characters	ASCII or Unicode

Important Concept: A binary pattern does not have a fixed meaning by itself. Its meaning depends on the representation system being used.

For example:

\[11111111 \]

It can represent:

- 255 in unsigned 8-bit representation.
- -1 in signed 8-bit two's complement representation.

The same binary bits can represent different values!

PART 2: Unsigned and Signed Integers

2. Unsigned Integers

Definition:

An unsigned integer is a binary number representation that can store only zero and positive numbers. It does not use a sign bit to represent negative values.

Explanation

In unsigned representation, all bits are used to represent the numerical value.

For an 8-bit unsigned number:

```
11111111
```

All eight bits contribute to the value.

Range of Unsigned Integers

The formula is:

\[\boxed{0\text{ to }2^n-1} \]

Where:

- \(n\) = Number of bits.
- Minimum value = 0.
- Maximum value = \(2^n-1\).

Example 1: 4-bit Unsigned Integer

Number of bits:

\[n=4 \]

Minimum:

\[0 \]

Maximum:

\[2^4-1=16-1=15 \]
Range: 0 to 15

Example 2: 8-bit Unsigned Integer

\[n=8 \]
\[Maximum=2^8-1 \]
\[=256-1=255 \]
Range: 0 to 255

Example 3: 16-bit Unsigned Integer

\[Maximum=2^{16}-1 \]
\[=65536-1=65535 \]
Range: 0 to 65535

Important Table

Bits	Minimum	Maximum
4-bit	0	15
8-bit	0	255
16-bit	0	65,535
32-bit	0	4,294,967,295
64-bit	0	\(2^{64}-1\)

3. Signed Integers

Definition:

A signed integer is a number representation that can store both positive and negative numbers as well as zero.

Explanation

Unlike unsigned representation, signed representation needs to indicate whether a number is positive or negative.

For this purpose, the most significant bit (MSB) is commonly used as the sign bit.

Sign Bit

The sign bit is the leftmost bit of a binary number that indicates its sign.

Sign Bit	Meaning
0	Positive
1	Negative

Example:

```
01010101
```

Sign bit = 0, so it represents a positive number in signed representation.

```
11010101
```

Sign bit = 1, so it represents a negative number in signed representation.

Important: This sign-bit interpretation is used in common signed formats, including two's complement, but the actual value depends on the encoding.

Types of Signed Integer Representation

There are three traditional methods:

1. Sign-Magnitude Representation
2. 1's Complement Representation
3. 2's Complement Representation

Let's understand each one.

PART 3: Sign-Magnitude Representation

4. What is Sign-Magnitude?

Definition:

Sign-magnitude is a signed number representation method in which the most significant bit represents the sign, while the remaining bits represent the magnitude (absolute value) of the number.

Rules

- 0 in the MSB represents positive.
- 1 in the MSB represents negative.
- Remaining bits represent the magnitude.

Example 1: Represent +5 in 8 bits

First convert 5 into binary:

$$ 5=0000101 $$

Add the positive sign bit 0:

$$ \boxed{00000101} $$

Example 2: Represent -5 in 8 bits

Magnitude of 5:

$$ 0000101 $$

Set sign bit to 1:

$$ \boxed{10000101} $$

Important: Sign-Magnitude Range

For an n-bit number:

$$ -(2^{n-1}-1)\text{ to }+(2^{n-1}-1) $$

It has two representations of zero: positive zero and negative zero.

For 8 bits:

$$ -127\text{ to }+127 $$

PART 4: 1's Complement

5. What is 1's Complement?

Very Important Numerical

Definition:

1's complement is a method of representing negative binary numbers by inverting every bit of the corresponding positive binary number.

Rule

- Change every 0 into 1.
- Change every 1 into 0.

In simple words, reverse all bits.

Example 1: Find 1's complement of 01010101

Original number:

```
01010101
```

Invert every bit:

```
10101010
```
Answer: 10101010

Example 2: Represent -5 using 1's complement in 8 bits

Step 1: Convert positive 5 into 8-bit binary.

$$ +5=00000101 $$

Step 2: Invert all bits.

```
Original:     00000101
1's Complement: 11111010
```
-5 = 11111010

Example 3: Find the 1's complement of 11001100

```
Original:       11001100
1's Complement: 00110011
```

Answer:

$$ \boxed{00110011} $$

Range of 1's Complement

For n bits:

$$ -(2^{n-1}-1)\text{ to }+(2^{n-1}-1) $$

For 8 bits:

$$ -127\text{ to }+127 $$

Important: 1's complement has two representations of zero:

- Positive zero: 00000000
- Negative zero: 11111111

PART 5: 2's Complement

6. What is 2's Complement?

Most Important Numerical

Definition:

2's complement is a method of representing signed binary numbers in which a negative number is obtained by inverting all bits of its positive binary representation and adding 1.

Formula

$$ \boxed{2's\ Complement=1's\ Complement+1} $$

Steps

1. Convert the positive number into binary.
2. Invert all bits (find 1's complement).
3. Add 1 to the result.

Example 1: Find 2's complement of 00000101

Step 1: Original number

```
00000101
```

Step 2: Invert all bits

```
11111010
```

Step 3: Add 1

```
  11111010
+ 00000001
-----------
  11111011
```

Answer: 11111011

This represents -5 in 8-bit two's complement.

Example 2: Represent -10 in 8-bit 2's complement

Step 1: Convert +10 into binary.

$$ +10=00001010 $$

Step 2: Find 1's complement.

```
11110101
```

Step 3: Add 1.

```
  11110101
+ 00000001
-----------
  11110110
```

-10 = 11110110

Example 3: Find 2's complement of 00110100

Step 1: Invert all bits.

```
11001011
```

Step 2: Add 1.

```
  11001011
+ 00000001
-----------
  11001100
```

Answer:

$$\boxed{11001100}$$

Why Is 2's Complement Important?

1. It represents both positive and negative integers.
2. It simplifies binary subtraction.
3. It uses only one representation of zero.
4. It makes arithmetic operations easier for CPU hardware.
5. It is the standard signed integer representation in modern computers.

PART 6: Integer Range and Comparison of Signed Representations

7. Range of Signed Integers

In n-bit two's complement representation, the range is:

$$\boxed{-2^{n-1}\text{ to }2^{n-1}-1}$$

Why?

- One bit is used to represent the sign.
- The remaining bit patterns are distributed between negative and non-negative values.
- There is one zero representation.

Example 1: 4-bit Signed Integer

$$n=4$$

Minimum:

$$ -2^{4-1}=-2^3=-8 $$

Maximum:

$$ 2^{4-1}-1=2^3-1=7 $$
Range: -8 to +7

Example 2: 8-bit Signed Integer

$$ n=8 $$

Minimum:

$$ -2^7=-128 $$

Maximum:

$$ 2^7-1=127 $$
Range: -128 to +127

Example 3: 16-bit Signed Integer

$$ n=16 $$

Minimum:

$$ -2^{15}=-32768 $$

Maximum:

$$ 2^{15}-1=32767 $$
Range: -32768 to +32767

Important Range Comparison

Bits	Unsigned Range	Signed 2's Complement Range
4	0 to 15	-8 to 7
8	0 to 255	-128 to 127
16	0 to 65,535	-32,768 to 32,767
32	0 to $2^{32}-1$	-2^{31} to $2^{31}-1$

Exam tip: If the question asks for signed range, use the two's complement formula unless another representation is specifically mentioned.

8. Comparison of Sign-Magnitude, 1's Complement and 2's Complement

Feature	Sign-Magnitude	1's Complement	2's Complement
Sign bit	MSB	MSB	MSB
Negative number	Change sign bit	Invert all bits	Invert and add 1
Positive number	Normal binary	Normal binary	Normal binary
Zero representations	Two	Two	One
8-bit range	-127 to +127	-127 to +127	-128 to +127
Common modern use	Limited	Limited	Standard

PART 7: Binary Subtraction Using 2's Complement

9. How to Perform Subtraction Using 2's Complement?

Definition:

2's complement subtraction is a technique in which subtraction is performed by adding the two's complement of the second number to the first number.

Formula

$$\boxed{A-B=A+(-B)}$$

Steps

1. Convert both numbers into binary using the same number of bits.
2. Find the 2's complement of the second number.
3. Add it to the first number.
4. If an extra carry is generated, discard it.
5. The remaining bits represent the answer.

Example 1: Calculate 7 − 3 using 4-bit representation.

Step 1: Convert both numbers into binary.

$$7=0111$$
$$3=0011$$

Step 2: Find 2's complement of 3.

Original:

```
0011
```

Invert:

```
1100
```

Add 1:

```
1100
+0001
-----
1101
```

Step 3: Add it to 7.

```
  0111
+ 1101
------
 10100
```

Step 4: Discard the extra carry.

```
0100
```

Answer: 0100 = 4

Therefore:

\[7-3=4 \]

Example 2: Calculate 5 − 8 using 8-bit representation.

Step 1: Convert to binary.

\[5=00000101 \]
\[8=00001000 \]

Step 2: Find 2's complement of 8.

```
Original: 00001000
Invert:   11110111
Add 1:    11111000
```

Step 3: Add.

```
  00000101
+ 11111000
-----------
```

```
  11111101
```

The MSB is 1, so the result is negative.

Step 4: Find its magnitude.

Find the 2's complement of 11111101:

```
Invert:  00000010
Add 1:   00000011
```

Magnitude = 3.

Answer: -3

Therefore:

\[5-8=-3 \]

PART 8: Overflow

10. What is Overflow?

Very Important Conceptual + Numerical

Definition:

Overflow occurs when the result of an arithmetic operation exceeds the range that can be represented using the available number of bits.

Explanation

Every binary representation has a limited range.

For example, an 8-bit signed two's complement number can represent:

\[-128\text{ to }+127 \]

If we perform an operation whose mathematical result is greater than 127 or less than -128, overflow occurs.

Example 1: Positive Overflow

Consider 8-bit signed addition:

$$ 100+50=150 $$

But the maximum 8-bit signed value is 127.

Convert both numbers:

$$ 100=01100100 $$
$$ 50=00110010 $$

Add:

```
  01100100
+ 00110010
-----------
  10010110
```

The result has MSB = 1, which appears negative in two's complement.

But the actual mathematical result is +150.

Therefore, overflow has occurred.

Overflow Occurred

Example 2: Negative Overflow

Consider:

$$ -100+(-50)=-150 $$

But the minimum 8-bit signed value is -128.

Binary representations:

$$ -100=10011100 $$
$$ -50=11001110 $$

Add:

```
  10011100
+ 11001110
-----------
  01101010
```

The result appears positive, even though both operands were negative.

Therefore, overflow occurred.

Rules for Detecting Signed Overflow

Memorize these rules:

1. Adding two positive numbers produces a negative result → Overflow.
2. Adding two negative numbers produces a positive result → Overflow.
3. Adding numbers with different signs → No signed overflow.

Important: A carry-out alone does not determine signed overflow.

Unsigned Overflow vs Signed Overflow

Unsigned Overflow	Signed Overflow
Occurs when the result exceeds the unsigned range.	Occurs when the result exceeds the signed range.
Usually detected through carry-out.	Detected by sign changes or the carry into/out of the sign bit.
Example: 255 + 1 in 8 bits.	Example: 127 + 1 in 8 bits.

Example 3: Unsigned Overflow

In 8-bit unsigned representation:

\[255+1=256 \]

But the maximum representable value is 255.

```
  11111111
+ 00000001
-----------
 100000000
```

Only 8 bits are available, so the extra carry is lost.

Stored result:

```
00000000
```

Unsigned overflow occurred.

PART 9: Sign Extension and Zero Extension

11. What is Sign Extension?

Definition:

Sign extension is the process of increasing the bit length of a signed binary number by copying its sign bit into the newly added higher-order bits.

Explanation

Suppose we have a 4-bit signed number:

```
1101
```

We want to convert it into an 8-bit representation.

The leftmost bit is 1, so we add four 1s to the left.

```
Before: 1101
After:  11111101
```

Both represent -3 in two's complement.

Example 1: Positive Number

Given:

```
0101
```

Sign bit = 0.

Extend to 8 bits:

```
00000101
```

Example 2: Negative Number

Given:

```
1010
```

Sign bit = 1.

Extend to 8 bits:

```
11111010
```

Important Rule

- If sign bit is 0, add zeros.

- If sign bit is 1, add ones.

12. What is Zero Extension?

Definition:

Zero extension is the process of increasing the bit length of an unsigned binary number by adding zeros to the left side.

Example 1

Given 4-bit number:

```
1010
```

Convert into 8 bits:

```
00001010
```

Example 2

Given:

```
0111
```

Convert into 8 bits:

```
00000111
```

Difference Between Sign Extension and Zero Extension

Sign Extension	Zero Extension
Used for signed numbers.	Used for unsigned numbers.
Copies the MSB into new positions.	Always adds zeros.
Preserves signed numerical value.	Preserves unsigned numerical value.
Example: 1101 → 11111101	Example: 1101 → 00001101

Important Exam Question

Q: Why is sign extension necessary?

Sign extension is necessary when increasing the bit width of a signed number because it preserves the original numerical value, including its sign.

PART 10: Fixed-Point Representation

13. What is Fixed-Point Representation?

Definition:

Fixed-point representation is a method of representing fractional numbers in which the binary point is placed at a fixed position within the number.

Explanation

In ordinary binary representation, we usually work with whole numbers.

However, computers also need to represent fractional values such as:

- 5.5
- 2.75
- 10.25

Fixed-point representation divides the available bits into two sections:

1. Integer part.
2. Fractional part.

The binary point remains at a fixed position.

Binary Fractional Place Values

The positions to the right of the binary point represent negative powers of 2.

Position	Weight
First	$2^{-1}=0.5$
Second	$2^{-2}=0.25$
Third	$2^{-3}=0.125$
Fourth	$2^{-4}=0.0625$

Example 1: Convert 101.101 into Decimal

Given:

$$ (101.101)_2 $$

Expand using positional weights:

$$ =1\times2^2+0\times2^1+1\times2^0 $$
$$ +1\times2^{-1}+0\times2^{-2}+1\times2^{-3} $$

$$ =4+0+1+0.5+0+0.125 $$
$$ \boxed{(101.101)_2=(5.625)_{10}} $$

Example 2: Convert 10.11 into Decimal

$$ (10.11)_2 $$
$$ =1\times2^1+0\times2^0 $$
$$ +1\times2^{-1}+1\times2^{-2} $$
$$ =2+0+0.5+0.25 $$
$$ \boxed{(10.11)_2=(2.75)_{10}} $$

Example 3: Convert 5.25 into Binary

Separate the number into integer and fractional parts.

Integer part:

$$ 5_{10}=101_2 $$

Fractional part:

$$ 0.25\times2=0.50 $$

Integer bit = 0.

$$ 0.50\times2=1.00 $$

Integer bit = 1.

Read bits from top to bottom:

$$ 0.25=0.01_2 $$

Combine:

$$ \boxed{(5.25)_{10}=(101.01)_2} $$

Advantages of Fixed-Point Representation

1. Simple hardware implementation.
2. Fast arithmetic operations.
3. Predictable precision.
4. Useful in embedded systems and digital signal processing.

Disadvantages

1. Limited range.
2. Fixed fractional precision.
3. Can cause overflow if values exceed the available range.
4. Less flexible for representing very large and very small numbers.

PART 11: Floating-Point Representation

14. What is Floating-Point Representation?

Very Important

Definition:

Floating-point representation is a method of representing real numbers, including very large, very small, and fractional values, using a sign, exponent, and significand (mantissa).

Explanation

Fixed-point representation has a fixed binary point.

Floating-point representation allows the binary point to move depending on the exponent.

For example, the same number can be represented in scientific notation:

\[6250=6.25\times10^3 \]

Similarly, binary floating-point uses a normalized form:

\[1.101\times2^3 \]

This makes it possible to represent a wide range of numbers.

Main Components of Floating-Point Representation

There are three major components:

3. IEEE 754 FLOATING POINT For Comp. ORG.pdf
slideshare.net

1. Sign Bit

Indicates whether the number is positive or negative.

- 0 = Positive

- 1 = Negative

2. Exponent

Determines the position of the binary point and represents the scale of the number.

3. Mantissa (Significand)

Contains the significant digits that represent the precision of the number.

General Formula

For a normalized binary floating-point number:

\[\boxed{(-1)^S\times1.M\times2^{E-Bias}} \]

Where:

- S = Sign bit.
- M = Fraction/mantissa bits.
- E = Stored exponent.
- Bias = Exponent bias.

PART 12: IEEE 754 Standard

15. What is IEEE 754?

Definition:

IEEE 754 is an international standard that defines how computers represent and perform arithmetic operations on floating-point numbers.

Why Is IEEE 754 Used?

1. Provides a standard format for floating-point numbers.
2. Supports positive and negative numbers.
3. Supports very large and very small values.
4. Defines precision and rounding rules.
5. Helps different computer systems handle floating-point numbers consistently.

IEEE 754 Formats

The two most commonly studied formats are:

1. Single Precision (32-bit).
2. Double Precision (64-bit).

16. Single Precision (32-bit)

In single precision, the total size is 32 bits.

Its structure is:

IEEE 754 Single Precision

1

Sign

8

Exponent

23

Fraction

Component	Number of Bits
Sign	1
Exponent	8
Fraction	23
Total	32

Bias of Single Precision

$$\boxed{Bias=127}$$

The actual exponent is calculated as:

$$Actual\ Exponent=Stored\ Exponent-127$$

Example

If the stored exponent is:

$$ E=130 $$

Then:

$$ Actual\ Exponent=130-127 $$
$$ \boxed{3} $$

17. Double Precision (64-bit)

Double precision uses 64 bits.

IEEE 754 Double Precision

1

Sign

11

Exponent

52

Fraction

Component	Number of Bits
Sign	1
Exponent	11
Fraction	52
Total	64

Bias of Double Precision

$$ \boxed{Bias=1023} $$

Actual exponent:

$$ Actual\ Exponent=Stored\ Exponent-1023 $$

Comparison of IEEE 754 Formats

Feature	Single Precision	Double Precision
Total bits	32	64
Sign bits	1	1
Exponent bits	8	11
Fraction bits	23	52
Bias	127	1023
Precision	About 7 decimal digits	About 15–16 decimal digits

PART 13: IEEE 754 Numerical Questions

18. Convert 5.75 into IEEE 754 Single Precision

Important Numerical

This is a common type of IEEE 754 question.

Question: Represent +5.75 in IEEE 754 single-precision format.

Step 1: Convert integer part into binary.

\[5_{10}=101_2 \]

Step 2: Convert fractional part into binary.

\[0.75\times2=1.50 \]

First bit = 1.

\[0.50\times2=1.00 \]

Second bit = 1.

Therefore:

\[0.75=0.11_2 \]

Combine:

\[5.75=101.11_2 \]

Step 3: Normalize the number.

Move the binary point so there is exactly one nonzero digit before it.

\[101.11=1.0111\times2^2 \]

Therefore:

- Sign = 0 (positive).
- Actual exponent = 2.
- Mantissa = 0111000... (remaining bits are zeros).

Step 4: Calculate the biased exponent.

For single precision:

\[Bias=127 \]
\[Stored\ Exponent=2+127=129 \]

Convert 129 into binary:

\[129=10000001_2 \]

Step 5: Write the fraction.

The normalized number is:

\[1.0111 \]

The leading 1 is implicit, so it is not stored.

Fraction:

`01110000000000000000000`

Step 6: Combine all fields.

Final IEEE 754 representation

0

Sign

10000001

Exponent

01110000000000000000000

Fraction

Complete 32-bit binary:

```
0 10000001 01110000000000000000000
```

Final Answer

19. Convert -10.5 into IEEE 754 Single Precision

Step 1: Sign bit

The number is negative.

$$ S=1 $$

Step 2: Convert magnitude into binary.

Integer:

$$ 10=1010_2 $$

Fraction:

$$ 0.5=0.1_2 $$

Therefore:

$$ 10.5=1010.1_2 $$

Step 3: Normalize.

$$ 1010.1=1.0101\times2^3 $$

Actual exponent = 3.

Step 4: Calculate stored exponent.

$$ 3+127=130 $$
$$ 130=10000010_2 $$

Step 5: Write fraction.

```
01010000000000000000000
```

Final Answer

```
1 10000010 01010000000000000000000
```

20. Special Values in IEEE 754

IEEE 754 also defines special values.

Value	Representation condition
Positive zero	Sign 0, exponent all zeros, fraction all zeros
Negative zero	Sign 1, exponent all zeros, fraction all zeros
Positive infinity	Sign 0, exponent all ones, fraction all zeros
Negative infinity	Sign 1, exponent all ones, fraction all zeros
NaN	Exponent all ones, fraction nonzero

NaN means Not a Number.

It is used for results that do not represent an ordinary numerical value, such as certain invalid floating-point operations.

Important: The normal formula with an implicit leading 1 does not apply to zero, subnormal values, infinity, and NaN.

PART 14: Fixed-Point vs Floating-Point

21. Important Comparison

Expected Theory Question

Fixed-Point	Floating-Point
Binary point has a fixed position.	Binary point can move.
Limited range.	Very large range.
Generally simpler hardware.	More complex hardware.
Precision depends on fixed fractional bits.	Precision depends on significand size.
Useful in embedded systems.	Useful in scientific calculations.
Example: 101.101	Example: \(1.01101\times2^2\)

Q: Which representation is better for very large and very small numbers?

Floating-point representation is generally better because it supports a much wider range of values.

PART 15: ASCII

22. What is ASCII?

Definition:

ASCII (American Standard Code for Information Interchange) is a character encoding standard that assigns numerical codes to letters, digits, punctuation marks, and control characters.

Explanation

Computers store everything in binary form, including text.

For example, when we type the letter A, the computer stores its corresponding numerical code.

In ASCII:

\[A=65 \]

Decimal 65 in binary:

\[65=01000001_2 \]

Therefore:

```
Character: A
Decimal:   65
Binary:    01000001
```

Important ASCII Values

Character	Decimal	Binary
A	65	01000001
B	66	01000010
C	67	01000011
a	97	01100001
b	98	01100010
0	48	00110000
1	49	00110001
Space	32	00100000

Characteristics of ASCII

1. Standard ASCII uses 7 bits.
2. It provides 128 possible codes (0–127).
3. It represents English letters, numbers, punctuation and control characters.
4. It is used for basic text representation.

5. Extended 8-bit ASCII variants use additional codes, but they are not one universal standard.

Example

Find the binary representation of character A.

ASCII value of A:

\[65 \]

Convert 65 into binary:

\[65=01000001 \]

Answer:

\[\boxed{A=01000001} \]

PART 16: Unicode

23. What is Unicode?

Definition:

Unicode is a universal character encoding standard designed to represent characters and symbols from almost all writing systems and languages around the world.

Explanation

ASCII mainly supports basic English characters.

However, computers also need to represent:

- English
- Urdu
- Arabic
- Chinese
- Japanese
- Mathematical symbols
- Emojis

Unicode provides a much larger collection of character codes to support these languages and symbols.

Characteristics of Unicode

1. Supports characters from different languages.
2. Supports mathematical and special symbols.
3. Supports emojis.
4. Provides unique code points for characters.
5. Is used by modern operating systems, applications and websites.

Examples of Unicode

Character	Unicode Code Point
A	U+0041
B	U+0042
ا (Arabic Alef)	U+0627
ع (Arabic Ain)	U+0639
€	U+20AC
😀	U+1F600

Unicode Encoding Formats

Unicode uses different encoding formats:

Format	Description
UTF-8	Uses 1–4 bytes per code point
UTF-16	Uses 2 or 4 bytes per code point
UTF-32	Uses 4 bytes per code point

Important: Unicode is the character standard, whereas UTF-8, UTF-16 and UTF-32 are encoding formats used to store Unicode code points.

24. Difference Between ASCII and Unicode

Very Important Theory Question

ASCII	Unicode
Originally designed for English text.	Supports characters from many languages.
Standard ASCII uses 7 bits.	No single fixed bit length.
Supports 128 codes.	Supports over a million possible code points.
Limited character set.	Much larger character set.
Does not directly support most international scripts.	Supports Urdu, Arabic, Chinese, emojis, etc.

Q: Why is Unicode better than ASCII?

Unicode supports a much wider range of characters and languages, whereas standard ASCII is limited to 128 codes and primarily supports basic English text.

PART 17: Important Numerical Questions — Complete Practice

Here are the numerical patterns you should be able to solve independently in the examination.

Topic	Practice Question
Unsigned range	Find the range of a 12-bit unsigned integer.
Signed range	Find the range of a 12-bit signed integer.
Sign-magnitude	Represent -18 using 8 bits.
1's complement	Find the 1's complement of 00110101.
2's complement	Find the 2's complement of 00010110.
Subtraction	Calculate 12 − 7 using 2's complement.
Overflow	Check overflow in 8-bit signed addition: 100 + 60.
Sign extension	Extend 1011 from 4 bits to 8 bits as a signed number.
Zero extension	Extend 1011 from 4 bits to 8 bits as an unsigned number.
Fixed-point	Convert 110.101 into decimal.
Floating-point	Represent 6.5 in IEEE 754 single precision.

Solved Numerical 1: Range of 12-bit Unsigned Integer

Formula:

$$0\text{ to }2^n-1$$

Put:

$$n=12$$
$$Maximum=2^{12}-1$$
$$=4096-1=4095$$

Answer:

$$\boxed{0\text{ to }4095}$$

Solved Numerical 2: Range of 12-bit Signed Integer

Formula:

$$ -2^{n-1}\text{ to }2^{n-1}-1 $$

Put:

$$ n=12 $$
$$ Minimum=-2^{11}=-2048 $$
$$ Maximum=2^{11}-1=2047 $$

Answer:

$$ \boxed{-2048\text{ to }2047} $$

Solved Numerical 3: Represent -18 Using 8-bit Sign-Magnitude

First convert 18 into binary:

$$ 18=00010010 $$

For sign-magnitude:

- Sign bit = 1 because the number is negative.
- Magnitude = 0010010 (7 bits).

Final:

$$ \boxed{10010010} $$

Solved Numerical 4: Find 1's Complement of 00110101

Original:

```
00110101
```

Invert every bit:

```
11001010
```

Answer:

$$ \boxed{11001010} $$

Solved Numerical 5: Find 2's Complement of 00010110

Step 1: Invert:

```
11101001
```

Step 2: Add 1:

```
  11101001
+ 00000001
-----------
  11101010
```

Answer:

\[\boxed{11101010} \]

Solved Numerical 6: Calculate 12 − 7 Using 2's Complement

Convert:

\[12=00001100 \]
\[7=00000111 \]

Find 2's complement of 7:

```
Original: 00000111
Invert:   11111000
Add 1:    11111001
```

Add:

```
  00001100
+ 11111001
-----------
 100000101
```

Discard extra carry:

```
00000101
```

Answer:

\[\boxed{5} \]

Solved Numerical 7: Check Overflow in 100 + 60

Using 8-bit signed representation:

\[100=01100100 \]

$$ 60=00111100 $$

Add:

```
  01100100
+ 00111100
-----------
  10100000
```

Both numbers are positive, but the result has sign bit 1.

Therefore:

Signed Overflow

Solved Numerical 8: Sign Extension

Given:

```
1011
```

Sign bit = 1.

Extend to 8 bits:

```
11111011
```

Answer:

$$ \boxed{11111011} $$

Solved Numerical 9: Zero Extension

Given:

```
1011
```

Add four zeros to the left:

```
00001011
```

Answer:

$$ \boxed{00001011} $$

Solved Numerical 10: Fixed-Point Conversion

Convert:

$$(110.101)_2=(?)_{10}$$

Solution:

$$=1\times2^2+1\times2^1+0\times2^0$$
$$+1\times2^{-1}+0\times2^{-2}+1\times2^{-3}$$
$$=4+2+0+0.5+0+0.125$$
$$\boxed{6.625}$$

PART 18: Important Expected Midterm Questions

Short Questions

1. Define data representation.
2. What is an unsigned integer?
3. What is a signed integer?
4. Define sign bit.
5. What is sign-magnitude representation?
6. Define 1's complement.
7. Define 2's complement.
8. Why is 2's complement preferred?
9. What is integer range?
10. Define overflow.
11. Differentiate between signed and unsigned overflow.
12. What is sign extension?
13. What is zero extension?
14. Define fixed-point representation.
15. Define floating-point representation.
16. What is IEEE 754?
17. What is the difference between single and double precision?
18. What is exponent bias?
19. Define ASCII.
20. What is Unicode?
21. What is the difference between ASCII and Unicode?
22. What are sign, exponent and mantissa?

Long Questions

1. Explain signed and unsigned integer representations with examples.
2. Explain sign-magnitude, 1's complement and 2's complement with solved examples.

3. Explain integer range for signed and unsigned numbers.
4. Explain overflow with suitable binary addition examples.
5. Explain sign extension and zero extension with examples.
6. Explain fixed-point and floating-point representation.
7. Explain IEEE 754 single and double precision formats.
8. Convert a decimal number into IEEE 754 single precision.
9. Differentiate between ASCII and Unicode.

PART 19: Week 4 Interactive Practice Test

Try these MCQs to check your preparation.

Week 4 — Midterm MCQs

0/12 attempted

Q1. What is the range of an 8-bit unsigned integer?

-128 to 127
0 to 255
-127 to 127
0 to 128

Q2. What is the range of an 8-bit signed two's complement integer?

0 to 255
-127 to 127
-128 to 127
-255 to 255

Q3. How do you calculate 2's complement?

Invert bits only
Add 1 only
Invert bits and add 1
Remove MSB

Q4. What is the 1's complement of 10101010?

10101010
01010101
11111111
00000000

Q5. What is the 2's complement of 00000101?

11111010
11111011
00000110
11110101

Q6. When does signed overflow occur?

Positive + negative gives positive
Positive + positive gives negative
Negative + positive always overflows
Whenever carry-out occurs

Q7. Which extension is used for signed numbers?

Zero extension
Sign extension
Bit shifting
Rotation

Q8. How many bits are used for the exponent in IEEE 754 single precision?

1
8
11
23

Q9. What is the exponent bias for IEEE 754 single precision?

64
127
255
1023

Q10. How many fraction bits are used in IEEE 754 double precision?

23
32
52
64

Q11. How many bits are in standard ASCII?

4
7
8
16

Q12. Which character encoding supports emojis and many international scripts?

ASCII
Binary only
Unicode
Sign-magnitude
Submit Answers

WEEK 04 — Final Revision Cheat Sheet

1. Integer Range Formulas

Unsigned:

$$ 0\text{ to }2^n-1 $$

Signed (two's complement):

$$ -2^{n-1}\text{ to }2^{n-1}-1 $$

2. Complement Rules

- 1's complement = Invert all bits.
- 2's complement = Invert all bits + 1.

3. Overflow Rules

- Positive + Positive = Negative → Overflow.
- Negative + Negative = Positive → Overflow.
- Different signs → No signed overflow.

4. Extension Rules

- Sign extension → Copy MSB.
- Zero extension → Add zeros.

5. IEEE 754

Feature	Single	Double
Total bits	32	64
Sign	1	1
Exponent	8	11
Fraction	23	52
Bias	127	1023

6. Character Encoding

- ASCII = Standard 7-bit encoding.
- Unicode = International character standard.
- UTF-8 = 1–4 bytes per code point.

My recommendation for tomorrow's midterm

Give these topics the highest priority:

Highest Priority

1. 1's and 2's complement numericals.
2. Signed and unsigned integer ranges.
3. Binary subtraction using 2's complement.
4. Overflow detection.
5. Sign extension and zero extension.
6. IEEE 754 single-precision conversion.

Then prepare fixed-point, ASCII, Unicode and the theory differences.

Important: Do not skip IEEE 754 numerical practice. Learn the exact sequence: convert to binary → normalize → determine sign → calculate biased exponent → write fraction → combine 32 bits.

This completes all the topics listed under Week 4 of your syllabus.